Quizzify Project Documentation

A browser-based local multiplayer quiz game built with HTML, CSS, and JavaScript.

1. Project Overview

Quizzify is a browser-based quiz game designed for solo play, multiplayer hot-seat play, and team-vs-team mode. It runs entirely in the browser and uses localStorage for saving selections and leaderboard scores.

2. Main Features

Category selection, difficulty levels, mode selection, custom player names, question count selection, countdown timer, scoring, streak system, sound effects, and high score storage.

3. Project Structure

The app is split into HTML pages for the home screen, gameplay screen, result screen, and leaderboard, with supporting CSS and JavaScript files for logic and styling.

4. How the Home Screen Works

The index page lets the user choose the category, difficulty, number of players, player names, and number of questions. These values are saved in localStorage and passed into the game session.

5. How the Game Works

The game page reads the saved settings, selects questions from the chosen category, starts a timer, and evaluates each answer. Score, streak, player turn, and progress are updated during play.

6. Difficulty and Timer

Easy = 18s, Medium = 13s, Hard = 10s. The timer decreases every second and triggers a warning when the countdown reaches 5 seconds.

7. Multiplayer Logic

The game tracks each player in an array and rotates turns after every question. Team mode treats the players as two teams, and the result screen identifies the highest total score.

8. Local Storage

localStorage stores category, difficulty, player list, game mode, question count, recent score, and saved leaderboard entries. This allows the app to remain fully static and local.

9. Question Data

Questions are stored in questions.json in a structured JSON format with category, question, four answer choices, and the correct answer index.

10. Result and High Score Flow

When the quiz ends, the app redirects to the result screen. For solo mode, the user can save a score to the leaderboard. The leaderboard page reads the saved results and displays them in order.

11. Run the App

Open index.html directly in a browser or run a local server such as: python -m http.server 8000 and then visit http://localhost:8000.

12. Conclusion

This project works as a lightweight, no-framework quiz game that is easy to run, modify, and customize. It demonstrates practical JavaScript logic, browser storage, UI flow, and game-state management.

Key files:

- index.html — configuration and home page
- game.html — quiz gameplay view
- game.js — question logic, timer, score tracking, multiplayer flow
- end.js — final result handling and saving scores
- highscores.js — leaderboard rendering
- questions.json — local question bank
- app.css — UI styling and animations

Developer Notes

This app was built as a fully front-end quiz project. It does not use a backend or database. The localStorage mechanism allows the app to retain settings and scores without external services. The game is intentionally lightweight so it can be opened and run without any setup or build tool.

Project status: Completed demo / local quiz game with multiplayer features.